Risk-Limiting Audits Are Interactive Proofs — How Do We Guarantee They Are Sound?

Paper Submission #147

***Abstract*—Risk-limiting audits (RLAs) are a central component of trustworthy elections. Although the statistical and logistical properties of RLAs have been studied and refined, much less attention has focused on modeling the underlying interactive protocol itself to show that a real-world malicious prover cannot deviate from the expected protocol to undermine the statistical guarantees. We model an RLA as an Interactive Proof and prove soundness against a prover bounded by polynomially many oracle queries. This gives us a rigorous basis for a general risk-limiting audit protocol, establishes minimal trust assumptions under which that protocol is indeed risk-limiting, and enumerates the aspects of the audit that must be observable by the public to confirm that the claimed risk limit was attained.**

1. Introduction

The need for trustworthy, evidence-based elections [1], [2] is increasingly obvious as election results are challenged around the world. A central component of an evidence-based election is a risk-limiting audit (RLA), which efficiently confirms correct outcomes statistically, and with high probability corrects erroneous outcomes before they become final. (Here "outcome" means who or what won, not the numerical vote totals.) In fact, an RLA is any procedure that has a known maximum probability of failing to correct erroneous outcomes; that maximum chance is the *risk limit*. Importantly, the procedure must carefully follow a precise protocol and a set of preconditions must be satisfied, otherwise it is impossible for the audit to guarantee a limit on the risk. Further, even if the authorities perfectly follow the RLA protocol, the public may not see enough evidence to be convinced.

The contribution of this paper is to model RLAs as interactive proofs, making assumptions and the verification algorithm explicit. We state and clarify several requirements and preconditions of RLAs previously unmentioned in the RLA literature and show that some conditions mentioned in the literature are not needed. We clarify what the public needs to see and check to have evidence that the claimed risk limit has been attained. We prove soundness of the underlying interactive protocol for the first time. Although there is prior work modeling the RLA protocol (rather than just the statistical methodology), we are the first to model the core security goal: limiting the risk that the verification algorithm accepts a wrong election result, even if the prover cheats.

An *interactive proof* [3] is a formal protocol between two parties: a computationally unbounded *prover* (in our case, the electoral authority) and a resource-bounded *verifier* (in our case, the public). Through this protocol, the prover seeks to convince the verifier that a statement is true (in our case, that the reported election outcome is the correct outcome). The interaction proceeds as a dialogue: the verifier issues randomized challenges, and the prover responds. Two properties are generally of interest: completeness and soundness. *Completeness* means that if the statement is true and the prover follows the protocol honestly, the verifier accepts with high probability. *Soundness* means that if the statement is false, no (even malicious) prover can convince the verifier to accept, except with a small probability (in our case, the risk limit plus a negligible extra).

The RLA setting has some important differences from the usual setting for cryptographic-style interactive proofs where an *instance* x belongs to a *language* $\mathcal{L}$. For RLAs, the instance x includes a trusted physical collection of paper ballot cards.[1] The prover has complete access to x but may make inaccurate claims about it; the verifier has partial access via queries. Querying this instance (i.e., retrieving a specific card for the verifier to inspect) is expensive, and a great deal of RLA research seeks to minimize the portion of x that must be inspected to attain a given level of confidence in the outcome (when the outcome is correct). However, it is feasible for the prover to record data about every card, and it is common the prover to provide a combination of *cast vote records (CVRs)* (electronic records purporting to be the voters' selections on individual cards) and reported vote subtotals for batches of cards. This data can be read efficiently by the verifier, but is not trusted. The language $\mathcal{L}$ is defined by the social choice function (also known as the electoral system), the reported winner, and additional technical details discussed below. For example, in a plurality contest where Alice is the reported winner, the language is something like, "all sets of at most N ballots containing more valid votes for Alice than for any other candidate."

The *reported outcome* is the outcome claimed by the prover. We define the *outcome* (or *correct outcome*) to be the true winner(s) according to the expressed preferences on every card in the instance. With an electoral authority or

1. We distinguish between *ballots* and *ballot cards* (or *cards*). A card is a piece of paper. A ballot includes all the contests a given voter is eligible to vote in. In some democracies there is only one contest per piece of paper, but in others (e.g., Switzerland and the USA) electors typically vote on many questions in one election, so a ballot comprises multiple cards.

team of auditors taking the role of prover, and the public taking the role of verifier, an oversimplified auditing process looks like this:

1. there is some election initialization and trusted setup,
2. the prover commits to the reported outcome (a set of reported winners), a set of ballot-card identifiers,[2] and auxiliary information (e.g., CVRs or batch subtotals),
3. the verifier requests some cards, addressing them by their identifier,
4. the prover retrieves some cards (possibly omitting some requested cards and/or including unrequested cards) and shows them to the verifier,
5. the verifier decides whether to accept the reported outcome at risk limit α.

The risk-limiting property is a *soundness* property: an upper bound on the probability that the verifier accepts the reported outcome (in Step 5) when the reported outcome is actually wrong, i.e., when the cards are not actually an instance of the language. We assume that if the verifier rejects, there will be a full manual tabulation of the instance, which will reveal the correct outcome (if one can be established).

1.1. Random Samples in Practice

In an adversarial setting, the problem of proving to observers that the cards were selected at random is a critical part of the audit. The challenge-response nature of the RLA protocol means that is is not enough for the samples to be random: they must also be unpredictable when the prover makes its first commitments. If the adversary knows in advance which ballots will be sampled (or the order in which ballots will be sampled), cheating is easy.

In practice, most RLAs require that after the electoral authorities have committed to the election data (outcome, tallies, CVRs, number of ballots, ballot manifest, etc), they use public, mechanical means to generate trustworthy randomness, for instance, by rolling dice. Some RLAs generate each selection separately, but this can rapidly become impractical for large samples (unless you have a lot of dice). Most use the randomness as a seed for a pseudorandom generator (`PRG`), which is then used to sample ballots. This means that, *once the seed is chosen*, the sequence of samples is known to the prover. We model this random seed as a challenge from the verifier.

Confusingly, RLAs in practice can use one or more "rounds" of sampling: at the end of each round, some data analysis based on the human reading of votes from the sampled cards informs the decision to accept the reported outcome or to examine more cards (possibly all cards). However, these separate rounds of sampling amount to a single round for the purposes of an interactive proof—the verifier issues only one challenge, which determines the entire sequence of samples. We therefore model the RLA as a one-round interactive proof, but this model subsumes RLAs that perform more than one round of sampling derived from a single seed.

2. The ballot-card identifiers can take various forms. For instance, the identifier may be a number printed on each card before or during scanning, or it might be the index of the card in a pile which is kept securely in the order in which it was scanned.

1.2. Related Work

Methods to conduct RLAs of plurality, multi-winner plurality, and super-majority contests were introduced in 2008 by [4], [5]. RLAs for other social choice functions have been developed over the years, including D'Hondt [6]; instant runoff voting (IRV) [7], [8], [9]; all scoring rules, approval voting, and Borda count [10]; small single transferable vote (STV) elections [11], [12]; and some Condorcet methods like ranked pairs [13]. Most of the literature has emphasized the statistical techniques, to minimize sample sizes, to simplify calculations (for transparency and reproducibility), or to develop RLAs for new settings (such as new social choice functions or complex sampling designs mandated by law, technology, or logistics). It has not formalized soundness at a protocol level against a malicious prover, though this is clearly the intended security goal.

The exceptions are [14], [15], which formalize various aspects of an RLA in an adversarial setting and prove the protocols to be risk-limiting. The main result of [14] is that for card-level comparison audits of plurality contests [16], it is not necessary to commit to every cast vote record (CVR) in advance. Rather, it suffices to commit to batch vote subtotals (the totals for each candidate within a well-defined batch of votes). Ballot selection then happens in two phases: first, some batches are chosen randomly, then the adversary is allowed to create CVRs for the batches that were chosen (subject to constraints that ensure that batch subtotals of the CVRs match the committed subtotals), then individual cards are sampled from the chosen batches. This result is surprising because it allows the prover to delay commitment to CVRs.

While [14] makes valuable contributions, it glosses over a crucial aspect of the audit process: the practicalities of verifiable random selection. It simply assumes that fresh, trustworthy randomness is used to generate the samples. In particular, Step 4 of the Basic Experiment simply says "Select a row [i.e., a CVR for audit] uniformly," and in the formal RLA auditing game the auditor requests individual ballots one at a time. The main result about delaying CVR commitment is proven only in this very strong model, limiting its practical applicability. The usual implementation of pseudorandom selection with a seed, described in Section 1.1, does not suffice—it is crucial to re-seed the pseudorandom generator *after* the new CVRs have been committed to. Otherwise, the adversary can simply cheat on the CVRs it knows will not be checked. This is implemented correctly in the sample code accompanying [14], but the formalization in the paper does not adequately model this process—a naïve implementation of the protocol might be completely insecure.

1.3. Our Contributions

In this paper, we model the RLA procedure and its surrounding details as an interactive protocol with an explicit verifier challenge used to deterministically generate the audited ballots. This facilitates proofs of soundness that approximate real scenarios more closely than prior work. In fact, our interactive protocol is more general than necessary for the RLA procedure, making it potentially applicable to various other scenarios.

Although our model does not incorporate multi-round interactive proofs with multiple challenges of the kind required by [14], we show that their proof fails in the single-challenge case, demonstrating that our formalism (unlike that of [14]) is expressive enough to identify this gap. The positive proof of security in the multi-challenge case is left as future work.

In Section 2, we specify the necessary assumptions of an RLA procedure and provide an overview of our main contribution: the interactive protocol and its soundness proof. In Section 3, we present the necessary background, and in Section 4, we define the interactive protocol. In Section 5, we prove that this protocol is sound. Finally, we conclude in Section 6.

2. Sound Proofs in Risk-Limiting Audits

Before introducing formalisms, we present an overview of the protocol and considerations specific to modeling risk-limiting audits as interactive proofs.

For the remainder of the paper, we assume we are auditing one contest in one election, unless stated otherwise. We begin with a trusted collection x of ballot cards and a *reported outcome* for a contest that appears on a subset of the cards. The prover P claims that a proper tabulation of the votes on the cards yields the reported outcome; the verifier V is checking this claim by requesting cards and examining the votes on those cards.

A prover could cheat by not returning the card that was requested—either by not returning any card or returning a different card. We therefore need to ensure that the verifier can tell whether the prover retrieves the requested ballots. This is solved in our model by insisting that each paper record has an indelible ID, which may in practice be a a number imprinted on the card or the position of the card in a batch in storage. Note, however, that the latter is actually malleable, a vulnerability in how some jurisdictions currently implement RLAs. A similar assumption is made in [14].

We also need to assume that there is some trustworthy upper bound—which does not come from the prover—on the total number of validly cast cards. Otherwise, a cheating prover could simply hide enough cards to make the remainder consistent with any reported outcome. (Depending on the audit strategy, we may also need a trustworthy upper bound on the number of cards that contain the contest.)

Usually, if the audit uses auxiliary data such as CVRs or other batch subtotals, the verifier checks that the auxiliary data yield the reported outcome. See Section 4.7.

Additional setup for an RLA. A common approach is to calculate a p-value for the hypothesis that the announced election outcome is wrong.[3] In an RLA, a threshold (the *risk limit*) α is set in advance. This can define the stopping condition if the p-value is computed in a sequentially valid way: P keeps sampling cards until the p-value of the null hypothesis falls below α, and the verifier V accepts only if (everything about the protocol has been conducted correctly and) V agrees that the p-value is below α. In a *risk-measuring audit*, the audit stops after examining any number of cards, and the audit reports the p-value based on the cards that were examined.

The following sections specify what the verifier must check in order to be satisfied that "everything about the protocol has been conducted correctly" and prove that it implies that the overall soundness error of the protocol is negligibly greater than α.

2.1. Pre- and Post-conditions for an RLA

The entirety of an RLA procedure cannot be modeled in the interactive proof framework; thus a sound interactive proof does not necessarily imply that an audit is risk-limiting unless a set of assumptions are satisfied, which we partition into pre-conditions and post-condtions. These must be checked and verified externally. If any of these do not hold, the procedure is not a risk-limiting audit but instead mere security theater. The pre-conditions are:

1) The (physical) ballot cards are an accurate (though not necessarily complete) record of eligible voters' true intentions.
2) There is a unique canonical way of interpreting any card and an accepted and deterministic process to resolve ambiguities and disputes.
3) We have a trustworthy (genuine) upper bound N_{card} on the total number of cards in the election event, and optionally also a trustworthy upper bound N_{contest} on the number of validly cast cards that contain the contest.
4) Whatever technology is used for ballot-card identifiers is "indelible" for the purpose of the audit, i.e., an identifier cannot be removed or altered once it has been committed. Moreover, identifiers cannot be added to cards after the commitment.

Note that we we explicitly allow the collection of ballot cards to be incomplete (Item 1). This is in contrast to prior work in the RLA literature, which unnecessarily requires that the collection is complete (see e.g., [17]). Further, we also explicitly highlight the need for indelible identifiers on cards (Item 4).

It is expected that the above pre-conditions are verified, at least informally, in practice. For example:

3. A p-value is any random variable whose distribution is dominated by the uniform when the null hypothesis is true. i.e., any random variable X such that $\Pr\{X \leq q\} \leq q \ \forall q \in [0,1]$ if the null hypothesis is true. If the p-value equals q, then either the null hypothesis is false, or an event occurred that had probability at most q. See Definition 13.

- Some jurisdictions conduct a *canvass* and *compliance audit* [2] to confirm that the collection of recorded votes includes only votes from eligible voters, with no cards added, substituted, or changed. Others allow observers to be present through all ballot handling and transport.
- Teams of observers may be able to keep track of the number of people who voted in person, offering an upper bound on the number of validly cast cards (the actual number of ballots may be lower because some voters may have left the polling place without casting their vote but after voter check-in, for example). It may be impossible for the public to verify the number of validly cast vote-by-mail ballots.
- The ballot-card retrieval process needs to allow observers to see that printed identifiers were not added/altered or, if identifiers are reference to a position in a pile, that the order of the cards was not rearranged.

For an audit to be a genuine RLA, some things must occur after the sampling and inspection, which we call the post-conditions:

5) If the risk limit is not met by sampling, the audit must lead to a full hand tabulation of the contest.
6) If the number of cards in that "full" tabulation that contain the contest is less than the upper bound on the number of cards that contain the contest, and the outcome could be different depending on the votes on those (possibly) "missing" cards, the audit shall declare that a correct outcome cannot be determined.
7) Otherwise, the audit must change the legal outcome to be the outcome according to the "full" tabulation. (Some jurisdictions purport to conduct RLAs, but their audits occur after the winners are certified or cannot legally change the reported outcome even when the audit determines that the reported outcome is wrong.)

Note that we allow the audit to declare that a correct outcome is indeterminable (Item 6), also contrary to prior work in the RLA literature; in practice, some of the instance x may be inaccessible for a "full" count.

Only Item 2, Item 3 and Item 4 will be directly used in our interactive proof, making RLAs a special case of our protocol.

2.2. The Interactive Proof

This paper focuses on the interactive proof that is part of the risk-limiting audit, and on proving that the interactive proof is sound under the assumptions stated above.

As is traditional in the cryptographic literature, we assume that the prover and the verifier know in advance the interactive protocol they are expected to execute. In particular, the election rules, including the card styles (i.e., the collection of contests that appear together on a ballot card) and the social choice function are known by everyone, as well as the chosen risk limit and pseudorandom generator (PRG). For some risk-limiting audit protocols, this will also include knowledge of the encryption algorithms, and zero knowledge (ZK) protocols that will be used.

The protocol is public coin, that is, the verifier sends all its random coins (random bits) to the prover, and the verifier eventually decides about the evidence provided by the prover using a deterministic algorithm that takes the full interaction as input. This means that the prover can decide from its own view whether the verifier will accept the election result based on the messages that are exchanged.

In order to limit the number of public coins that are required, risk-limiting audits typically generate a limited number of random bits, which are then expanded using a PRG, which we model as a random oracle. Our prover can only make a bounded number of queries to that random oracle.

2.3. Protocol Overview

Proof statement. The prover starts by choosing the statement about which it wishes to complete the proof, and makes that statement public. The statement contains the following:

- A commitment to a (multi-)set of ID-tagged ballot cards. This commitment is typically a physical process: ballot cards are placed in custody, possibly after having all been marked with a permanent identifier (which is expected to be unique, but might not be). This commitment is expected to be perfectly binding.[4]
- A perfectly binding commitment to a set of IDs that may or may not be the IDs of the committed ID-tagged ballot cards. The prover is *supposed to* use unique IDs and report them accurately, but a dishonest (or incompetent) prover might re-use IDs or omit IDs or misreport the IDs that it used.
- A reported outcome and a set of assertions about the committed ID-tagged ballot cards. These assertions are expected to imply the reported outcome; that is, if the assertions are true, the reported outcome must be correct.
- A risk limit for the proof.
- Optionally, auxiliary but untrusted data that the verifier may choose to use during the audit process, including tallies, subtotals, or (possibly encrypted) cast vote records (CVRs).
- A trustworthy (genuine) upper bound on the number of ballot cards.
- A statement on whether the audit will be *style-based*[5] or not, and, if so a trustworthy upper bound on the number of ballot cards that contain the audited contest (for style-based sampling).

The prover does not have the option of selecting invalid upper bounds for either the overall cards or the cards containing the contest. In practice, this requires a trustworthy upper bound to come from outside the system being audited.

4. It is not important whether any of these commitments are hiding.

5. Style-based sampling simply means that we use the card style information to limit our sampling to only draw ballot cards that (we believe) contain the contest of interest. Note that the card style information is not necessarily trustworthy, just as with CVRs.

Challenge. The verifier publicly selects a random seed for a PRG uniformly from $\{0,1\}^\lambda$, where λ is the length of the seed in bits.

Response. The prover derives a random sequence of ballot IDs from the seed using the PRG and returns a sequence of ID-tagged ballot cards from the multiset that it committed to in the first step. The prover chooses the length of the sequence of ID-tagged ballot cards that it returns, and whether they match or do not match the sequence of ballot IDs derived from the seed expansion.

The verifier's decision. The verifier decides in a deterministic way whether to accept the returned sequence of ID-tagged ballot cards as sufficient evidence that the reported contest outcome is right.

Soundness of the proof. The protocol is sound if the probability that the verifier accepts a reported outcome that is inconsistent with the full committed multiset of ballot cards is at most negligibly higher than the risk limit.

The rest of the paper develops a proof that this soundness result holds, under the assumptions stated above.

3. Definitions and Background

A *ballot* is the physical evidence of one voter's choices. A ballot may comprise multiple *cards*, each of which may or may not contain the contest in question. We begin with the notation we use in this paper.

- $[a, b]$ is the set of *real numbers* in the range from a to b inclusive.
- $a..b$ is the set of *integers* in the range from a to b inclusive.
- $y \leftarrow z$ means that variable y is assigned value z.
- $y \underset{\text{rnd}}{\leftarrow} Z$ means that y is an element of the set Z selected uniformly at random from Z.
- N_{card} is an upper bound on the total number of validly-cast cards in the election.
- N_{contest} is an upper bound on the total number of validly-cast cards that contain the contest.
- $\mathcal{V}$ is the set of all possible valid votes, which always includes the *null vote* NULL, which indicates every card that contains the contest but the voter did not make a valid choice.
- BAD designates any invalid vote value, for example if the contents of the card are disputed, or if the ID returned does match what was requested.
- NOCONTEST designates any card in the election that does not contain the contest in question.
- $\mathcal{I}$ is a set of available IDs, of size at least N_{card}.

3.1. Ballots, Cards, CVRs and Valid Interpretations

Notation 1. *A* Cast Vote Record (CVR) *is an electronic record purporting to be the vote on one card. CVRs are not necessarily accurate representations of the votes on the card.*[6]

Notation 2. *A* Manual Vote Record (MVR) *is the vote on a card according to an accurate manual inspection of the card.*

For the contest under audit, a CVR or MVR either

1) contains a (possibly invalid) vote in the contest (i.e., an element of $\mathcal{V} \cup \{\text{BAD}\}$), or
2) does not contain the contest, which is represented by NOCONTEST.

3.2. Social Choice Functions and Election Outcomes

A *social choice function* maps votes to an election outcome. An outcome need not be a single winning candidate—it could be a set of winning candidates, an allocation of seats to parties, etc.

Definition 3. Let $\mathcal{O}$ be a set of possible election outcomes and $\mathcal{V}$ be the set of valid votes. A *social choice function* TALLY maps a multiset of elements of $\mathcal{V} \cup \{\text{BAD}\}$ to an element of $\mathcal{O}$, or $\perp$ if the outcome cannot be determined.

Implicit in this definition is that the outcome depends only on the multiset of votes cast, and not, for instance, on the order in which votes are cast or tabulated.

We allow the social choice function to input BAD votes, because sometimes the number of disputed or missing votes may be so high that the outcome cannot be determined.

3.3. What We Are Trying to Prove

An instance x is a multiset of tuples consisting of cards and their corresponding IDs. IDs cannot be changed or added once the audit starts. A tuple $(\perp, \mathit{card})$ denotes a card with no ID, for instance, because an ID was maliciously or mistakenly not printed.

Let $\mathcal{L}$ be the set of all multisets of ID-tagged ballot cards that TALLY maps to the reported outcome and let $\mathcal{A}_U$ be the set of all multisets of ID-tagged ballot cards that respect the stated upper bound (N_{card} on the total number of cards and optionally also N_{contest} on the cards containing the audited contest, if it is being used).

We will prove a *soundness* property: if x respects the assumptions but produces a different outcome, i.e., if $x \in \overline{\mathcal{L}} \cap \mathcal{A}_U$, then no malicious prover can convince the verifier that $x \in \mathcal{L}$ except with probability smaller than α (plus some negligible error—see below).

We cannot in general guarantee a *correctness* property: even if $x \in \mathcal{L} \cap \mathcal{A}_U$ and if the prover and verifier honestly follow the protocol, then sometimes the verifier may not accept the proof. For example, if N_{contest} is much larger than the number of available cards, the correct election outcome

6. Of course, in practice CVRs can contain data for multiple contests, but we consider only one at a time.

may be impossible to determine from all the available evidence.

The prover is allowed to choose x, but note that this has nothing to do with the authenticity of the voting cards, or with the security of the chain of custody. A proper RLA must still satisfy Item 1 (of Section 2.1), which is not part of our protocol, and must, as such, be checked externally. As a result, our protocol is more general than an RLA procedure.

The language $\mathcal{L}_{O,\mathcal{O},\text{TALLY}}$ is the set of all multisets such that each multiset consists of card-ID pairs that would, when interpreted canonically and assigned to problem categories in BAD when necessary, and then properly counted according to TALLY, produce result $O \in \mathcal{O}$.

We will define a protocol $\text{RLA-}\Pi_{\alpha,G_{\text{SAMPLE},N},\text{P-VAL}}$ for a prover P and a verifier V so that for all poly-time provers P, the probability that V accepts P's output when $x \in \overline{\mathcal{L}_{O,\mathcal{O},\text{TALLY}}} \cap \mathcal{A}_U$ is at most $\alpha + negl(\lambda)$, where λ is a chosen security parameter.

3.4. Assertions

An *assertion* is a predicate on multisets of votes, which may include BAD ones and those that do not contain the contest. In order to be useful for auditing, the predicate must satisfy two additional properties, which amount to sanity conditions on the kinds of social choice function we can audit. Adding cards that do not contain the contest does not change the assertion; adding unmarked votes does not turn the assertion from false to true.

Definition 4. An *assertion* A is a predicate on multisets of $\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\}$, satisfying two conditions:

1) **NOCONTEST Invariance:** For any vote multiset $\mathcal{M}$ and any multiset $\mathcal{N}$ of NOCONTEST votes,

$$A(\mathcal{M}) \iff A(\mathcal{M} \cup \mathcal{N})$$

2) **NULL Monotonicity:** For any vote multiset $\mathcal{M}$ and any multiset $\mathcal{N}$ of NULL votes,

$$A(\mathcal{M}) \implies A(\mathcal{M} \cup \mathcal{N})$$

Examples of assertions are "Alice received more votes than Bob," (for a plurality contest), "The yes votes meet the 60% threshold regardless of what the missing votes say," (for a proposition that requires a supermajority), and (for IRV) "When all candidates other than Chuan and Diego are eliminated, Diego has more votes than Chuan."

3.5. Connecting Assertions and Election Outcomes

The interactive proof will verify assertions, but we need a way for the verifier to check whether those assertions imply that the reported election outcome is the correct outcome. For some social choice functions, this obviously exists. For example, in a simple three-candidate plurality election we have $\mathcal{O} = \{$"Alice," "Bob," "Chuan"$\}$ and a social choice function TALLY given by "whoever gets the most votes wins." Suppose Alice is the reported winner. The assertion set $\{$"Alice has more votes than Bob"$\}$ does not imply that Alice won, as it says nothing about the third candidate, but if every assertion in the set

$$\{ \text{"Alice has more votes than Bob"}, \text{"Alice has more votes than Chuan"} \}$$

holds, Alice indeed won.[7]

In general, it may not be computationally feasible to decide whether a particular set of assertions (predicates over multisets of $\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\}$) imply a particular election outcome given a social choice function. This computation is feasible for the natural cases that we are aware of, but we make the assumption explicit here for two reasons:

- there may not be any efficient way of validating a set of assertions for some social choice functions, and
- it is important in practice to conduct the verification—otherwise even if the prover passes the audit of the proffered assertions, that is not evidence that the reported election outcome is correct.

Definition 5. An *assertion validation function* for a social choice function TALLY inputs

- an outcome $O \in \mathcal{O}$ and
- a set of assertions ASSERTIONS,

and outputs $\begin{cases} \text{true}, & \text{if } O \text{ is the unique outcome implied by} \\ & \text{ASSERTIONS given TALLY} \\ \text{false}, & \text{otherwise.} \end{cases}$

A false output might mean either that the assertions do not imply a unique outcome or that they imply a different outcome from O. We assume that the verifier has an efficient assertion validation function for the social choice function used in the election.

3.6. Committing to Information About Cards

The prover makes several commitments to untrusted data about the cards under audit—these are slightly different depending on whether the prover has chosen *style-based* or *non-style-based* auditing.

First, P must commit to which ID will be retrieved when a given value is sampled. For *non-style-based sampling* this must specify an ID for any value up to the card upper bound N_{card}; for *style-based sampling* only values up to the contest upper bound N_{contest} are used.

Definition 6. The prover's *ID commitment* is a function

$$\text{FIND} : 1..N_{\text{card}} \to \mathcal{I} \text{ for non-style-based sampling,}$$

or

$$\text{FIND} : 1..N_{\text{contest}} \to \mathcal{I} \text{ for style-based sampling.}$$

The prover may lie about the identifiers that are assigned to the voting cards—we make no assumptions about the

7. Even for plurality contests, there can be subtleties. For example, if ties are resolved alphabetically then it suffices to claim that Alice's total was *at least as high* as the others'; if ties are resolved in some other way an assertion of a *strictly higher* tally may be needed.

accuracy of its claims. We will prove that the audit remains sound even if:

- some cards were not assigned IDs in $\mathcal{I}$,
- some IDs the prover claimed to use were not in fact assigned to any physical card, or
- more than one card was assigned the same ID.

Verifying the basic consistency properties of FIND (e.g., that it is injective) is part of the verification algorithm—see Section 4.7 for details.

The prover is required to specify an ID for every index up to the upper bound. In particular, if the bounds are not tight (for example, if the prover cannot find all N_{card} cards) then the prover is expected to assign IDs to the missing values anyway, even if it knows no card can be found. (In practice this padding might happen in a latter stage after P indicates that no corresponding ballot exists, e.g., in SHANGRLA [10], this is implemented by padding the list with "phantoms" until the total that are available for sampling matches the appropriate upper bound.[8])

In card-level comparison audits, the prover makes an ID commitment and a *CVR commitment* in which it commits to a reference vote for some or all IDs in the ID commitment.

Definition 7. The prover's *CVR commitment* is a function

$$\mathit{CVR} : 1..N_{\text{card}} \to (\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\})$$

for non-style-based sampling, or

$$\mathit{CVR} : 1..N_{\text{contest}} \to (\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\})$$

for style-based sampling.

Definitions 6 and 7 together model the prover's initial declaration about the cards. The prover is declaring that if the sampling function chooses i, the prover will retrieve a card with ID FIND(i), and that applying TALLY to the set of CVRs in the commitment will produce the same outcome as applying TALLY to x. Of course, these two tables need to be checked for consistency with each other and with the rest of the election data. In particular, the verifier should check that applying TALLY to the CVRs yields the reported outcome. See Section 4.7 for details.

In the case of style-based sampling, the prover is implicitly making a claim that the cards containing the contest are the ones in the image of FIND. (Card style information can reduce audit sample sizes by orders of magnitude [18], [19].)

3.7. Assorters

This section draws primarily from the SHANGRLA framework developed in [10].

An assorter takes as inputs:

- a Manual Vote Record (*MVR*) (possibly a BAD or NOCONTEST one) for the sampled card,
- a *CVR* (possibly a BAD or NOCONTEST one).

8. See https://github.com/pbstark/SHANGRLA/blob/main/shangrla/core/Audit.py#L628 make_phantoms method (accessed 12 Nov 2025)

Some audits (*ballot-polling audits*) calculate results based only on the uploaded MVRs—the CVR input is not used in that case; others (e.g., *card-level comparison audits*) involve the difference between some function of the MVR and the same function of the CVR.

Definition 8. An *assorter* is a function

$$a : (\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\})^2 \to [0, u_a].$$

where $a(c, m)$ is the assorter value of MVR m and CVR c.

The assorter's upper bound u_a depends on the social choice function, the auditing strategy (e.g., comparison audit versus polling audit), and possibly the reported votes. See [10]. An assorter translates votes (and optionally CVRs) into bounded nonnegative numbers.

Many assertions, including the examples above, can be expressed in terms of the mean value of an assorter applied to x: the assertion is true iff the mean of the assorter is greater than 1/2. For instance, the assertion "Alice received more votes than Bob" is equivalent to the claim that the mean value of an assorter that takes the value 1 for cards that have a valid vote for Alice, the value 0 for cards that have a valid vote for Bob, and the value 1/2 otherwise, is greater than 1/2. Checking whether the assertion holds then amounts to testing the complementary hypothesis that the mean value of the assorter applied to all the validly cast cards is not greater than 1/2. To reject that hypothesis is to confirm the assertion. The other example assertions above also can be translated into the claim that the mean of a bounded list of nonnegative numbers is greater than 1/2.

Assorters need to deal conservatively with problems such as missing ballots and unmatched IDs; in general, defining the assorter to have the value 0 in such cases will be conservative, but in some cases less extreme values are also conservative. See [10, §3.4] and [20, §3].

An assorter expresses an assertion if the assertion must be true whenever the assorter's mean over x is greater than $1/2$.

Definition 9. Assorter a *expresses* assertion A under specific CVR commitments *CVR* when, for all possible sequences of votes $(\mathit{MVR}(i))_{i=1}^{N_{\text{card}}}$ drawn from $\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\}$,

$$\begin{aligned} &\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} a(\mathit{CVR}(i), \mathit{MVR}(i)) > 1/2 \\ &\implies \quad A\left(\{\mathit{MVR}(i)\}_{i=1}^{N_{\text{card}}}\right) \end{aligned} \tag{1}$$

This equation characterizes the essential correctness condition for an assorter—it says, "for any possible set of votes on the cards, if we were to apply the assorter to every card, the mean of the resulting list of numbers is greater than $1/2$ only if the assertion is true." This equation does not involve any sampling or probability—it is established by an argument about the assorter calculation and how it corresponds to the meaning of the assertion.

Appendix A contains assorters for both ballot polling and ballot-level comparison audits (Definition 23 and Definition 26 respectively), with proofs that they satisfy Definition 9.

Equation (1) is complicated to verify for comparison audits, because then the assorter depends on the CVRs. Roughly speaking, in a plurality contest where Alice reportedly beat Bob, if the mean of the *overstatement assorter* [10, §3.2]) for Alice and Bob is greater than 1/2, the net error in the CVRs favors Alice by less than the number of votes by which Alice's reported total exceeded Bob's reported total—so Alice really did receive more votes than Bob. The overstatement assorter assigns the value 0 to cards for which the CVR says 'Alice' but the MVR says 'Bob,' and takes its maximum value when the CVR says 'Bob' but the MVR says 'Alice.' See [10].

3.7.1. Assorters for style-based sampling. *Style-based sampling* is an auditing strategy that focuses only on cards that ought to contain the contest. It is important to understand that these strategies do *not* sample only from the ballots that the prover claims to contain the contest, since the prover's claims about style are untrusted, but rather from N_{contest} ballots. (N_{contest} is the trustworthy upper bound on the total number of cards that contain the contest.) Assorters for style-based sampling must take a worst-case assorter value if the card does not in fact contain the contest.

When sampling uses style information, the correctness condition for assorters has to be slightly modified.

Definition 10. Assorter a *expresses-with-style* assertion A under specific CVR commitments *CVR* when, for all possible sequences of votes $(MVR(i))_{i=1}^{N_{\text{card}}}$ drawn from $\mathcal{V} \cup \{\text{BAD}, \text{NOCONTEST}\}$ that respect the contest upper bound N_{contest}, for all index sets I of size N_{contest},

$$\begin{aligned} &\frac{1}{N_{\text{contest}}} \sum_{i \in I} a(CVR(i), MVR(i)) > 1/2 \\ &\implies \quad A\left(\{MVR(i)\}_{i=1}^{N_{\text{card}}}\right) \end{aligned} \tag{2}$$

Note the different indices in different parts of Equation (2): the assertion must be true *across the whole set of cards* whenever the assorter's mean is greater than $1/2$ *across any subset of size* $N_{contest}$. This means the assorter must deal conservatively with lies about style. In practice, the audit will chose I to include all the cards the prover claims to contain the contest, but the proof must remain sound even if the prover misrepresents that set.

Assorter constructions for style-based sampling are also given in Appendix A, together with proofs that each construction satisfies Definition 10.

3.7.2. Assumptions about assorters. Assorters need to deal with BAD ballots as the "worst case". For *MVR*s, this is formalized as follows:

$$\forall c, m \in \mathcal{V} \cup \{\text{BAD}\}, a(c, \text{BAD}) \leq a(c, m) \tag{3}$$

(A BAD vote can not produce a higher assorter value than any specific valid (or BAD) vote.)

Similarly, if the audit uses *style-based sampling,* the assorters need to deal with bad style data in the "worst case" as well; see [18], [19]. This is formalized as follows.

$$\forall c, m \in \mathcal{V} \cup \{\text{BAD}\}, a(c, \text{NOCONTEST}) \leq a(c, m) \tag{4}$$

3.8. Sampling Strategies

A sampling strategy describes the intended probability distribution for card samples.

Definition 11. A *sampling strategy* SAMPLE—parameterised by a population size N—is a family of probability distributions over (possibly infinite) sequences of values in $1..N$.

(N is expected to match either the card upper bound N_{card} or the contest upper bound N_{contest}.)

The sampling strategy specifies whether the sample may contain repeats (sampling with replacement versus without replacement), whether indices are sampled with equal probability (uniform or weighted sampling), whether cards are partitioned into disjoint strata from which samples are drawn independently (stratified or unstratified sampling), and whether individual cards or groups of cards are selected (whether the sampling unit is an individual card or a cluster or cards, such as all cards cast in a particular precinct).

Whenever we define a sampling strategy SAMPLE, we assume that there exists an associated polynomial time deterministic sampling function $\bar{G}_{\text{SAMPLE},N}$.

Definition 12. A ballot sampling function $\bar{G}_{\text{SAMPLE},N}$ for a sampling strategy SAMPLE is a family of deterministic polynomial time functions parameterised by a population size N which map ℓ uniformly random bits to a sequence of values in $1..N$ distributed according to SAMPLE_N. That is

$$\bar{G}_{\text{SAMPLE},N} = \text{SAMPLE}_N.$$

3.9. Statistical Functions

A p-value calculator inputs a sequence of values calculated by an assorter from votes. Assuming that the vote indices were selected using some specific sampling strategy SAMPLE, it assesses the hypothesis that the mean of the population (of assorter values) is less than or equal to $1/2$. Examples implemented in the SHANGRLA[9] library include ALPHA [21], Kaplan-Kolmogorov [10], Kaplan-Markov [22], and Wald's SPRT [23].

Definition 13. Suppose $x = (x_1, \ldots, x_N)$ satisfies $x_i \in [0, u]$ for all i, where u is known. A *p-value calculator* P-VAL for a given sampling strategy SAMPLE is a family of functions, indexed by t, u and N, that take

- a finite sequence of t values $(x_{i_j})_{j=1}^{t}$, where $(i_j)_{j=1}^{t}$ are drawn from $1..N$ using SAMPLE_N, and the value t is specified in advance of drawing the sample,

9. https://github.com/pbstark/SHANGRLA

- (optionally) untrusted auxiliary information AUX-P,

such that,

$$\forall t, u, N, \text{AUX-P}, p \in [0,1],$$
$$\Pr\{\text{P-VAL}_{u,N,t}(x_{i_1}, \ldots, x_{i_t}, \text{AUX-P}) \leq p\} \leq p$$

when the following hypothesis holds:

$$\frac{1}{N}\sum_{i=1}^{N} x_i \leq 1/2.$$

The auxiliary information AUX-P might be derived from the prover's untrusted claims about tallies and other data. For many p-value calculators this is unnecessary, but for those that use adaptive strategies, e.g., [9], [21], this information is useful for reducing the sample size required to attain a given p-value.

A P-VAL is *sequentially valid* if it will return a valid p-value even if the sample size at which to calculate it is not chosen in advance. For example, if it is evaluated repeatedly over a telescoping sample, the chance that it is *ever* below p is at most p.

Definition 14. A p-value calculator P-VAL for a given sampling strategy SAMPLE is *sequentially valid* if for all sequences of values (x_{i_j}), where (i_j) is drawn using the strategy SAMPLE_N,

$$\forall u, N, \text{AUX-P}, p \in [0,1],$$
$$\Pr\left\{\inf_t \text{P-VAL}_{u,N,t}(x_{i_1}, \ldots, x_{i_t}, \text{AUX-P}) \leq p\right\} \leq p$$

whenever the following hypothesis holds:

$$\frac{1}{N}\sum_{i=1}^{N} x_i \leq 1/2.$$

Our results assume a sequentially valid p-value calculator. This allows us to prove security for an audit in which the prover is allowed to stop at any time. Alternatively, one could fix the sample size and prove security *without* sequential validity. This follows from a slight modification to our main proof (Sec. 5), which we omit from this paper.

We also assume that P-VAL is monotone. Technically, monotonicity is not implied by Definition 13, but some of the p-value calculators used in existing RLAs are monotone, such as Wald's SPRT [23]. Monotonicity makes it easy to reason about how changes to P-VAL's inputs affect its output. Our main proof (Sec. 5) relies on this.

Definition 15. A p-value calculator P-VAL is *monotone* if for all u, N, t and all $x_1 \leq y_1, \ldots, x_t \leq y_t$, then $\text{P-VAL}_{u,N,t}(x_1, \ldots, x_t) \geq \text{P-VAL}_{u,N,t}(y_1, \ldots, y_t)$.

3.10. Pseudorandom Sampling with Random Oracles

In an RLA, it is typically impractical to perfectly implement a sampling strategy in a way that is publicly observable. The practical solution is to instead sample a small *seed* $s \in \{0,1\}^\lambda$ in a publicly observable way (e.g., with repeated and public dice throws) and then expand that seed using a pseudorandom generator $\texttt{PRG} : \{0,1\}^\lambda \to \{0,1\}^\ell$. That is, one samples s and evaluates

$$G_{\text{SAMPLE},N}(s) \triangleq \bar{G}_{\text{SAMPLE},N}(\texttt{PRG}(s))$$

where $\bar{G}_{\text{SAMPLE},N}$ is the SAMPLE's sampling function (§3.8).

We model PRG as a random oracle—as a function sampled uniformly at random before the RLA begins, that all parties can query [24]. Working in this model, we later prove that seed-based sampling does not meaningfully reduce the soundness of our RLA proof. To be clear, if λ is too small, then the prover can easily cheat. For example, if $\lambda = 1$, then the prover can pre-compute $G_{\text{SAMPLE},N}(0)$ and $G_{\text{SAMPLE},N}(1)$ and safely misreport any ballot not in these sequences. However, for computationally limited provers, and large λ, this kind of attack becomes infeasible. For example, if $\lambda = 128$ and the prover can make at most 2^{80} PRG queries, then the seed that will be sampled during the risk-limiting audit will result in a completely fresh sample with probability at least $1 - 2^{-48}$.

In practice, the random oracle PRG is instantiated using a cryptographic function such as AES, SHA2, or SHA3, as is standard [24].

4. Defining the RLA Game

Elections can have different social choice functions, requirements for a vote to be valid, and other rules for interpreting voters' marks and selections, requiring the statement and the proof to be parameterized for the specific situation. We will model two different kinds of parameters. First, *language parameters*, which specify the claim that is being made, that is, the language $\mathcal{L}$. Second, *proof parameters*, which model the kind of proof that is being offered. Put another way, language parameters define the election and its result, while proof parameters define the audit.

4.1. Language Parameters

The language parameters are:

- a set $\mathcal{O}$ of possible outcomes,
- a set $\mathcal{V}$ of valid votes, which must include NULL,
- a social choice function TALLY (Definition 3).

4.2. Proof Parameters

The proof parameters are:

- A risk limit $\alpha \in [0,1]$
- A sampling function $\bar{G}_{\text{SAMPLE},N}$ (see Section 3.8).
- A p-value calculator P-VAL (see Section 3.9).
- (Optionally) details for other cryptographic algorithms, if using, such as
  - the commitment parameters, (e.g., for VAULT [25])
  - ZK proof algorithms (e.g., for ElectionGuard [26] or VAULT aggregate tally decryption).

The risk limit, PRG, ballot sampling function, risk measuring function and cryptographic algorithms are typically decided in advance and implemented in software, but there is no inherent reason why this is necessary. It could also make sense for the verifier to choose them, or even the prover (provided that the chosen algorithms satisfy the requirements phrased earlier). Still, we consider that they must be determined and made public before the election: our security properties guarantee that the audit is sound even if the prover can choose his cheating strategy as a function of the parameters and algorithms that the verifier will use.

4.3. The Instance

Before the protocol begins, the prover fixes an *instance*, which represents the claim it will prove. The instance is:

- a multiset x of ID-tagged cards,
- a *reported outcome* $O \in \mathcal{O}$,
- a trustworthy upper bound N_{card} on the number of cards,
- (optionally) a decision to use style-based sampling and a trustworthy upper bound N_{contest} on the number of cards containing the contest.

The prover will attempt to show that the social choice function TALLY yields the reported outcome O, when applied to the cards x. The upper bounds are trusted in the sense that the RLA only protects against invalid instances in which x respects the bound indicated by the style choice. Our soundness definition (Definition 17) will capture this limitation.

Notation. Let N be N_{contest} if style-based sampling is selected, and N_{card} otherwise.

4.4. The Prover's First Message

In the first step of the protocol, the prover P commits to a collection of claims about x.

- An ID commitment FIND (see Definition 6).
- (Optionally) a *CVR* commitment (see Definition 7).
- A list ASSERTIONS of assertions with corresponding assorters ASSORTERS (see Definition 4, Definition 8 and Definition 9).
- (Optionally) other data AUX-P, such as tallies or subtotals, which the p-value calculator may input.

4.5. The Verifier's Challenge

The verifier V samples and sends a seed $s \underset{\text{rnd}}{\leftarrow} \{0,1\}^\lambda$.

Remark. In our model, V sends the random *seed* s rather than the pseudorandom sample π that is derived from s. This is cryptographically unusual, but it is how most RLAs work. Since RLAs aim to convince the public of an election's validity, most RLA verifiers take all their actions in public view. Thus, the prover can also see these actions—and the seed s. It is unusual to send a PRG seed to an adversary, but we are still able to prove security in the random oracle model (Sec. 5).

4.6. The Prover's Response

The prover P computes $\widetilde{s} \leftarrow \texttt{PRG}(s)$ and $\pi \leftarrow \bar{G}_{\text{SAMPLE},N}(\widetilde{s})$. For each index $i = 1..$, P either retrieves a (possibly BAD) ballot card and shows it to the verifier, or states that no corresponding ballot card will be retrieved by sending $\perp$. P stops when the sequence of indices has been entirely processed, or at any earlier arbitrary time.[10] This process makes the stopping point $t_{\max}$ and the sequence $\mathit{MVR}(i)_{i=1..t_{\max}}$—the unique canonical content of each retrieved card, possibly a BAD or NOCONTEST one—available to both P and V.

(P is supposed to retrieve $\mathit{MVR}(i)$ with identifier FIND(π_i) but is free to do otherwise.)

4.7. The Verifier's Decision

The verifier's algorithm has two possible outputs: ACCEPT or NON-ACCEPT. ACCEPT means that the verifier accepts the proof and consequently also the reported election outcome. NON-ACCEPT should lead to further action, which is outside the scope of this paper—see Section 2.1.

4.7.1. Validating the prover's first message. This can be done either immediately or at the end.

If any of the following checks fails, the whole audit outputs NON-ACCEPT. (In practice the Prover is allowed to fix it if validation fails before it learns the seed, but not afterwards.)

Validating the assertions and assorters

1) Each assorter satisfies its upper bound.
2) ASSERTIONS implies O given TALLY. (This assumes the existence of an efficient assertion validation function, Definition 5.)
3) $|\text{ASSERTIONS}| = |\text{ASSORTERS}|$.
4) for all $i \in 1..|\text{ASSORTERS}|$, ASSORTERS_i expresses ASSERTIONS_i, that is, satisfies Definition 10 for style-based sampling, or Definition 9 for non-style-based sampling, for the given *CVRs*.[11]

Validating the FIND functions

5) FIND assigns some $i \in \mathcal{I}$ to every index in $1..N$ and does not repeat any ballot ID.

4.7.2. Validating the prover's response to s. The validation process is detailed in Algorithm 1. The verifier V has the prover P's chosen stopping point $t_{\max}$. It iterates over the cards returned by P.

On Line 7, the verifier V identifies the identifier of the card that must be audited. On Line 8, V reads that card, or sets it to $\perp$ if no card has been returned.

On Line 10, V checks whether the card returned by the prover has the expected ID. If so, V sets MVR_t to the canonical interpretation of that card, and to BAD otherwise.

10. Note that P is not allowed to delay indefinitely, though it is allowed to retrieve nothing for any index.

11. This effectively says that if we are using style-based sampling and there are *CVR*s that claim to have the contest but are outside the image of FIND then we ignore them.

V then goes through all the assorters and, on Line 15, it extends the vector $\vec{a}_i$ with the value that assorter a_i takes for this t^{th} audited card. It also updates p_i with the p-value computed for this assorter on Line 16.

V can break early if all p-values are less than the risk limit—this is tested on Line 19, p-values for all assorters.

When the sequence of ballot samples is finished, the verifier V accepts only if all the p-values are below the risk limit α, as tested on Line 24.

Algorithm 1 Verifying P's response

1: $\widetilde{s} \leftarrow \texttt{PRG}(s)$
2: $\pi \leftarrow \bar{G}_{\text{SAMPLE},N}(\widetilde{s})$
3: **for** $t = 1..t_{\max}$ **do**
4: **if** $t \geq |\pi|$ **then** ▷ π is finite and finished.
5: **break**
6: **end if**
7: $id_t \leftarrow \text{FIND}(\pi_t)$.
8: $card_t \leftarrow$ the t-th card returned by P (or $\perp$)
9: **if** $card_t \neq \perp$ and id_t is the ID on $card_t$ **then**
10: $MVR_t \leftarrow$ the MVR for $card_t$
11: **else**
12: $MVR_t \leftarrow$ BAD
13: **end if**
14: **for** each assorter a_i **do**
15: $\vec{a}_i \leftarrow \vec{a}_i.\texttt{append}(a_i(CVR(\pi_t), MVR_t))$
16: $p_i = \text{P-VAL}_{u_i,N,t}(\vec{a}_i, \text{AUX-P})$
17: where u_i is the upper-bound on a_i
18: **end for**
19: **if** $\forall i, p_i < \alpha$ **then**
20: **break**
21: **end if**
22: **end for**
23: **for** each p-value p_i **do**
24: **if** $p_i \geq \alpha$ **then**
25: **return** NON-ACCEPT
26: **end if**
27: **end for**
28: **return** ACCEPT

4.8. Security Definition

We can now phrase the soundness guarantee that we expect.

Definition 16. The RLA-Π game, parameterized by a risk limit α, a sampling function $G_{\text{SAMPLE},N}$, a p-value calculator P-VAL and a security parameter λ, is the protocol described in Sections 4.4 to 4.7.

Definition 17. $\text{RLA-}\Pi_{\alpha,G_{\text{SAMPLE},N},\text{P-VAL}}(\lambda)$ is *sound* w.r.t. the language $\mathcal{L}$ parameterized by a social choice function TALLY and a reported outcome O from a set $\mathcal{O}$ of possible outcomes if, for every probabilistic prover P^* interacting with an honest verifier V in the RLA-Π game (Definition 16) there is an upper bound $\alpha + \text{negl}(\lambda)$ on the probability that V returns ACCEPT if x is in $\bar{\mathcal{L}} \cap \mathcal{A}_U$.[12]

5. Results

Our main result is contained in Theorem 18.

Theorem 18. *If* (P-VAL, SAMPLE) *satisfies Definitions 14 and 15 (a monotone and sequentially valid p-value calculator) and $G_{\text{SAMPLE},N}$ satisfies Definition 12 (i.e., implements* SAMPLE*) then for all $\alpha \in [0,1]$,* $\text{RLA-}\Pi_{\alpha,G_{\text{SAMPLE},N},\text{P-VAL}}(\lambda)$ *is is sound in the random oracle model—that is, it satisfies Definition 17—wrt language $\mathcal{L}_{O,\mathcal{O},\text{TALLY}}$.*

Our proof is a hybrid argument that uses an intermediate, idealized game. In the ideal game $\mathcal{G}_I$, the verifier V samples and sends a large, uniform $\widetilde{s} \in \{0,1\}^l$. This ignores the practical constraint that V cannot sample large strings.

Definition 19 (Ideal game $\mathcal{G}_I$)**.** $\mathcal{G}_I$ is the same as the RLA-Π game, except that (1) V samples and sends $\widetilde{s} \in \{0,1\}^l$ as its first message. (2) V skips computing $\widetilde{s}$ (line 1, Algorithm 1), and (3) P also skips computing $\widetilde{s}$.

We state two intermediate lemmas, then give their corresponding proofs in the following subsections. Let P^* be a PPT algorithm and $W(P^*, \mathcal{G})$ be the event that V outputs ACCEPT at the end of an interaction with P^* in the security game $\mathcal{G}$ on a multiset of cards in $\bar{\mathcal{L}} \cap \mathcal{A}_U$ and let $P_{\text{win}}(P^*, \mathcal{G}) = \Pr[W(P^*, \mathcal{G})]$.

The first lemma is that in $\mathcal{G}_I$, no algorithm can fool V with probability better than α.

Lemma 20. *For any (even unbounded) P_I^*,*

$$P_{win}(P_I^*, \mathcal{G}_I) \leq \alpha.$$

The second lemma states that $\mathcal{G}_I$ and the RLA-Π game are similar, in the following sense. For any P_R^* in the RLA-Π game, one can build a P_I^* for $\mathcal{G}_I$, with similar probability of winning.

Lemma 21. *Model* PRG *as a random oracle, and assume assume that* (P-VAL, SAMPLE) *satisfies Definitions 14 and 15. For any P_R^* that plays the* RLA-Π *game and queries at most* $\text{poly}(\lambda)$ *times, there is a P_I^* such that*

$$P_{win}(P_R^*, \text{RLA-}\Pi) - \text{negl}(\lambda) \leq P_{win}(P_I^*, \mathcal{G}_I).$$

Theorem 18 is an immediate corollary of these lemmas. We will now prove them.

5.1. The ideal game is sound

Now, we prove Lemma 20: with perfect sampling, V is fooled with probability at most α.

12. One can view $\log_2 \alpha$ as a *statistical* security parameter. In the cryptographic literature, a computational security parameter (λ) characterizes an error probability that can be increased through an adversary's computational work, while a statistical security parameter characterizes an error probability that cannot be increased in this way [27].

Proof. First, fix a card multiset x_0 that respects the upper bound but not the outcome O. That is, fix $x_0 \in \overline{\mathcal{L}_{O,\mathcal{O},\text{TALLY}}} \cap \mathcal{A}_U$. Second, fix an adversary P_0^* for $\mathcal{G}_I$, with V denoted V_0.

Our proof of Lemma 20 is itself a hybrid argument. We will define a sequence of games, each comprising (P_i^*, V_i, x_i). Let $p_{H,i}$ be the probability that P_i^* convinces V_i on cards x_i. We will show that $p_{H,0} \leq p_{H,1} \leq \cdots \leq \alpha$. Each new hybrid simplifies the previous, until the upper bound α is easy to show. Unless otherwise noted, each P_{i+1}^*, V_{i+1}, and x_{i+1} is respectively equal to P_i^*, V_i, and x_i.

Hybrid 1. In this hybrid, the verifier checks only one assertion. Since the outcome is wrong, some assertion A_f (with assorter a_f) is false. Let V_1 check only A_f. Since this is more permissive than V_0, $p_{H,0} \leq p_{H,1}$.

Hybrid 2. In this hybrid, we specify the response of the prover. P_2^* differs from P_1^* in only its response. For each request π_t, P_2^* replies with a card with an ID matching $\text{FIND}(\pi_t)$ and an MVR m that maximizes $a(\mathit{CVR}(\pi_t), m)$, or with $\perp$ if no such card exists. Since V_2 treats $\perp$ as a BAD MVR, by Equation (3), this choice maximizes all entries of $\vec{a}_f$ (Step 15 of Alg. 1). So, by Definition 15, p_f does not increase in V_2's computation. Thus, V_2 accepts whenever V_1 does, so $p_{H,1} \leq p_{H,2}$.

Hybrid 3. In this hybrid, we specify the number of cards. Let x_3 be x_2 with $N_{\text{card}} - |x_2|$ copies of $(\perp, \text{NOCONTEST})$ added. Since these additions have ID $\perp$, they will never be in P_3^*'s response (see hybrid 2), so we have $p_{H,2} = p_{H,3}$.

Hybrid 4. In this hybrid, we erase IDs that are not in FIND. Let x_4 be x_3 with any card $(\mathit{id}_i, \mathit{MVR}_i)$ replaced by $(\perp, \mathit{MVR}_i)$ if id_i is not in the image of FIND. Since the changed card had ID $\perp$, it is not queried by V_4, so we have $p_{H,3} = p_{H,4}$.

Hybrid 5. In this hybrid, we deduplicate the valid IDs on the cards. Let x_5 be x_4 with the following changes, repeated to fixpoint. First, let indices $i \neq j$ have MVRs MVR_i and MVR_j, and equal IDs $\mathit{id}_i = \mathit{id}_j \neq \perp$. Second, choose $k \in \{i, j\}$ to minimize $a_f(\text{FIND}^{-1}(\mathit{id}_k), \mathit{MVR}_k)$. Third, change id_k to $\perp$. Since k was chosen to minimize a_f, the MVR MVR_k is never sent to V_5, per the prover's response strategy (see hybrid 2). So, $p_{H,4} = p_{H,5}$.

Hybrid 6. In this hybrid, we replace $\perp$ IDs with unused, real ones. Let x_6 be x_5, but with each card $(\perp, \mathit{MVR}_i)$ replaced with $(\mathit{id}_i, \mathit{MVR}_i)$, where id_i is an ID not yet in x_6. Giving the card an unused ID modifies one entry of $\vec{a}_f$, if that ID is sampled. Since the ID was unused, V_5 would have sent $\perp$ for that ID (hybrid 2), and now it sends a card. So, per the same argument as in hybrid 2, $p_{H,5} \leq p_{H,6}$.

Hybrid 7. In this hybrid, we have the prover commit to IDs that are on cards. Let P_7^* send the same FIND and *CVR*s as P_6^*, with the following changes. Let I be the set of IDs in x_7. Note that $|I| = N_{\text{card}} \leq |\text{FIND}|$. For each i such that $\text{FIND}(i)$ is not on some card, change $\text{FIND}(i)$ an entry of I that is not in the image of FIND, and change $\mathit{CVR}(i)$ to the corresponding MVR. If V_7 queried the old $\text{FIND}(i)$, P_7^* would have returned $\perp$, and now they return some card. So, per the same argument as in hybrids 2 and 6, $p_{H,6} \leq p_{H,7}$.

The final bound, via statistics. We have constructed a new game (P_7^*, V_7, x_7) such that $p_{H,0} \leq p_{H,7}$. Furthermore, in this game: there are N_{card} cards, every entry of FIND and *CVR* matches a unique card, and P_7^* answers every query with that unique card. Thus, P_7^*'s behavior has been sufficiently constrained for us to analyze $p_{H,7}$ using non-interactive, statistical techniques. We will now do that.

Let N be either N_{contest} or N_{card}, according to whether P_7^* uses style-based sampling or not. Then, by Definition 9 or 10 and the falsity of A_f, we have

$$\frac{1}{N} \sum_{i=1}^{N} a_f(\mathit{CVR}(i), \mathit{MVR}(\text{FIND}(i))) \leq 1/2. \tag{5}$$

According to Steps 19 and 24 of Algorithm 1, V_7 accepts only if for some subsequence $\vec{a} = (a_f(\mathit{CVR}(\pi_i), \mathit{MVR}(\text{FIND}(\pi_i))))_{i=1}^{k}$:

$$\text{P-VAL}(\vec{a}, \text{AUX-P}) \leq \alpha$$

where π is sampled according to SAMPLE. By the sequential validity of P-VAL for SAMPLE (Definition 14), the probability of this event when Equation (5) holds is at most α. So, $p_{H,0} \leq p_{H,7} \leq \alpha$, which concludes our proof. □

5.2. The PRG creates negligible soundness loss

In this subsection, we prove Lemma 21: using PRG only reduces soundness by a negligible amount.

Proof. Recall that RLA-Π is like $\mathcal{G}_I$, except that a PRG is used for sampling (see Definition 19). Let P_R^* be the adversary for RLA-Π, with verifier V_R. Recall that P_R^* is defined relative to the random oracle PRG. Assume that it queries PRG at most once at each input. This is without loss of generality.

We define P_I^*, an adversary for $\mathcal{G}_I$, with verifier V_I, as follows. P_I^* sends the same initial message as P_R^*, waits for P_R^* to get s from V_R, and waits to get $\widetilde{s}$ from V_I. During this process, queries to PRG receive uniformly random responses, as normal. But then, P_I^* programs PRG at s: $\text{PRG}(s) \leftarrow \widetilde{s}$, waits for P_R^*'s response to V_R, and sends the same response to V_I.

We will now analyze $P_{\text{win}}(P_I^*, \mathcal{G}_I)$. Let C ('cheatable') be the set of seeds at which P_R^* queries PRG before its first message. By the uniformity and independence of s, we have $\Pr[s \in C] = |C|/2^\lambda \leq \text{negl}(\lambda)$. So P_I^*'s chance of winning is at most negligibly smaller than

$$\Pr[W(P_I^*, \mathcal{G}_I) \wedge s \notin C]$$

Let us analyze this probability. Since $s \notin C$, P_R^* wins the RLA-Π game if and only iff P_I^* wins $\mathcal{G}_I$. Thus we have

$$\begin{aligned}\Pr[W(P_I^*, \mathcal{G}_I)] &\geq \Pr[W(P_I^*, \mathcal{G}_I) \wedge s \notin C] - \text{negl}(\lambda) \\ &= \Pr[W(P_R^*, \text{RLA-}\Pi)] - \text{negl}(\lambda),\end{aligned}$$

as desired. □

6. Conclusion and Future Work

In this paper, we described a cryptographic interactive protocol that includes the RLA procedure. In doing so, we stated, clarified, and refined assumptions and conditions on RLAs that has been gone unmentioned or has been over-restrictive in the RLA literature. We then proved the soundness property of our interactive protocol and connect it to the risk-limiting property of RLAs.

As our protocol is quite general, we note that it is likely applicable to many other auditing scenarios, including financial audits, that should be done as real proofs instead of blindly trusting the auditors.

References

[1] P. Stark and D. Wagner, "Evidence-based elections," *IEEE Security and Privacy*, vol. 10, pp. 33–41, 2012, https://www.stat.berkeley.edu/~stark/Preprints/evidenceVote12.pdf.

[2] A. Appel and P. Stark, "Evidence-based elections: Create a meaningful paper trail, then audit," *Georgetown Law Technology Review*, vol. 4.2, pp. 523–541, 2020, https://georgetownlawtechreview.org/wp-content/uploads/2020/07/4.2-p523-541-Appel-Stark.pdf.

[3] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM J. Comput.*, vol. 18, no. 1, pp. 186–208, 1989.

[4] P. B. Stark, "Conservative statistical post-election audits," *The Annals of Applied Statistics*, vol. 2, no. 2, pp. 550 – 581, 2008.

[5] P. Stark, "A sharper discrepancy measure for post-election audits," *Ann. Appl. Stat.*, vol. 2, pp. 982–985, 2008. [Online]. Available: http://arxiv.org/abs/0811.1697

[6] P. Stark and V. Teague, "Verifiable european elections: Risk-limiting audits for d'hondt and its relatives," *JETS: USENIX Journal of Election Technology and Systems*, vol. 3.1, 2014. [Online]. Available: https://www.usenix.org/jets/issues/0301/stark

[7] M. Blom, P. J. Stuckey, and V. J. Teague, "Ballot-polling risk limiting audits for irv elections," in *Electronic Voting: Third International Joint Conference, E-Vote-ID 2018, Bregenz, Austria, October 2-5, 2018, Proceedings 3*. Springer, 2018, pp. 17–34.

[8] M. Blom, P. J. Stuckey, and V. Teague, "Raire: Risk-limiting audits for irv elections," *arXiv preprint arXiv:1903.08804*, 2019.

[9] A. Ek, P. B. Stark, P. J. Stuckey, and D. Vukcevic, "Adaptively weighted audits of instant-runoff voting elections: Awaire," in *International Joint Conference on Electronic Voting*. Springer Nature Switzerland Cham, 2023, pp. 35–51.

[10] P. Stark, "Sets of half-average nulls generate risk-limiting audits: SHANGRLA," *Financial Cryptography and Data Security, Lecture Notes in Computer Science*, vol. 12063, 2020, preprint: http://arxiv.org/abs/1911.10035.

[11] M. Blom, P. J. Stuckey, V. Teague, and D. Vukcevic, "A first approach to risk-limiting audits for single transferable vote elections," in *International Conference on Financial Cryptography and Data Security*. Springer, 2022, pp. 366–380.

[12] ——, "Rlas for 2-seat stv elections: Revisited," in *International Conference on Financial Cryptography and Data Security*. Springer, 2024, pp. 3–17.

[13] ——, "Risk-limiting audits for condorcet elections," in *International Conference on Financial Cryptography and Data Security*. Springer, 2023, pp. 79–94.

[14] A. Harrison, B. Fuller, and A. Russell, "Adaptive risk-limiting ballot comparison audits," *2023 IEEE Symposium on Security and Privacy (SP)*, pp. pp. 2002–2019, 2023. [Online]. Available: https://arxiv.org/abs/2202.02607

[15] B. Fuller, R. Pai, and A. Russell, "The decisive power of indecision:{Low-Variance}{Risk-Limiting} audits and election contestation via marginal mark recording," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5467–5484.

[16] P. Stark, "Super-simple simultaneous single-ballot risk-limiting audits," in *Proceedings of the 2010 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections (EVT/WOTE '10)*. USENIX, 2010. [Online]. Available: http://www.usenix.org/events/evtwote10/tech/full_papers/Stark.pdf

[17] P. B. Stark, "Overstatement-net-equivalent risk-limiting audit: Oneaudit," in *International Conference on Financial Cryptography and Data Security*. Springer, 2023, pp. 63–78.

[18] A. Glazer, J. Spertus, and P. Stark, "More style, less work: card-style data decrease risk-limiting audit sample sizes," *Digital Threats: Research and Practice*, vol. 2, pp. 1–15, 2021.

[19] ——, "Stylish risk-limiting audits in practice," in *E-VOTE-ID 2023*. Cham: Springer, 2023.

[20] P. Stark, "Non(c)esuch ballot-level risk-limiting audits for precinct-count voting systems," in *Computer Security. ESORICS 2022 International Workshops. Lecture Notes in Computer Science,13785*, S. e. a. Katsikas, Ed. Cham: Springer, 2023, pp. 541–554.

[21] P. B. Stark, "ALPHA: Audit that learns from previously hand-audited ballots," *Annals of Applied Statistics*, vol. 17, pp. 641–679, 2023.

[22] P. Stark, "Risk-limiting post-election audits: P-values from common probability inequalities," *IEEE Transactions on Information Forensics and Security*, vol. 4, pp. 1005–1014, 2009.

[23] M. Lindeman, P. Stark, and V. Yates, "BRAVO: Ballot-polling risk-limiting audits to verify outcomes," in *Proceedings of the 2011 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections (EVT/WOTE '11)*. USENIX, 2012.

[24] M. Bellare and P. Rogaway, "Random oracles are practical: a paradigm for designing efficient protocols," in *CCS*, 1993.

[25] J. Benaloh, P. B. Stark, and V. Teague, "Vault: Verifiable audits using limited transparency," in *Electronic Voting: Fourth International Joint Conference, E-Vote-ID 2019*, October 2019.

[26] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach, "ElectionGuard: a cryptographic toolkit to enable verifiable elections," in *33rd USENIX Security Symposium, USENIX Security 2024*. USENIX Association, 2024.

[27] Y. Lindell and B. Pinkas, "An efficient protocol for secure two-party computation in the presence of malicious adversaries," in *EUROCRYPT*, 2008.

[28] J. Bañuelos and P. Stark, "Limiting risk by turning manifest phantoms into evil zombies," arXiv.org, Tech. Rep., 2012, retrieved 17 July 2012. [Online]. Available: http://arxiv.org/abs/1207.3413

7. Ethics considerations

None.

Appendix

This section shows how to generate assorters that deal conservatively with various kinds of problem ballots, assuming as a starting point a way of scoring a (valid) vote. Suppose we have a function s that is somewhat like an assorter for assertion A, except that it takes only a single input (an MVR or CVR) and is defined only on valid votes. The function s outputs a real number in the range $[0, u_s]$.

There are some standard, general patterns for converting this sort of function into an assorter that expresses A and

takes both valid and BAD votes. The treatment in this section is equivalent to the "phantoms to zombies" approach developed in [10], [17], [28].

First we need to make precise what properties s should start with. For this, we need a variant of Equation (1).

Definition 22. Let s be a function from $\mathcal{V}$ to $[0, u_s]$ s.t. $s(\text{NOCONTEST}) = 1/2$. Then s is a *scoring function* for assertion A when, for all possible sequences of valid votes $(V(i))_{i=1}^{N_{\text{card}}}$ drawn from $\mathcal{V}$,

$$\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} s(V(i)) > 1/2 \Rightarrow A((V(i))_{i=1}^{N_{\text{card}}}) \tag{6}$$

This is intended to formalize the idea that s would be a valid assorter expressing assertion A if it ignored style information and CVRs, and only ever received valid ballots.

This section contains examples of valid ways to transform a function that satisfies Equation (6) for assertion A into a valid assorter that expresses A, by dealing conservatively with BAD input ballots and other problems.

There are two special values used repeatedly when making assumptions about missing data:

NOTHING The assorter value that corresponds to having not counted this value at all is $1/2$. This is appropriate for missing CVRs, which were not included in the reported tally.

WORST The assorter value least favorable to the assertion's reported winner is 0. This is appropriate for cards that have been lost, cards that have not been produced when required, or cards with the wrong id.

These special values give us general strategies for transforming a scoring function (which deals only with valid ballots) into one that deals conservatively with problem ballots, for either ballot-polling or ballot-level comparison audits.

For ballot-polling, there are no *CVR*s, so the assorter ignores its second input.

Definition 23. Suppose that s is a function from $\mathcal{V}$ to $[0, u_s]$. Define the phantoms-to-zombies transformation for polling, $a^{\text{PZ}}_{\text{POLL}} : (\mathcal{V} \cup \text{BAD}) \times (\mathcal{V} \cup \text{BAD}) \to [0, u_s]$, as follows.

$$a^{\text{PZ}}_{\text{POLL}}(c, m) = \begin{cases} s(m) & \text{if } m \in \mathcal{V} \\ 0 & \text{otherwise.} \end{cases}$$

This makes a valid assorter that ignores *CVR*s and deals conservatively with BAD ballots.

Proposition 24. *If s satisfies Definition 22 for assertion A, then $a^{\text{PZ}}_{\text{POLL}}$ as defined in Definition 23 is an assorter that expresses A. That is, it satisfies Equation (1) over $\mathcal{V} \cup \text{BAD}$. They have the same upper bound.*

Proof. We prove the contrapositive. Suppose that $a^{\text{PZ}}_{\text{POLL}}$ does not satisfy Equation (1). Then there must be specific values of *CVR* and *MVR* s.t. $A\left(\{MVR(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false but the LHS of the implication is true, so:

$$\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} a^{\text{PZ}}_{\text{POLL}}(CVR(i), MVR(i)) > 1/2.$$

Suppose there is some $v \in \mathcal{V}$ s.t. adding more of v to the MVRs cannot cause A to switch from false to true.

Consider a modified list $\widetilde{MVR}$ of *MVR*s in which v replaces all BAD values. By assumption, $A\left(\{\widetilde{MVR}(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false. We will show that $\widetilde{MVR}$ makes the left side of Equation (6) true. From above:

$$\begin{aligned}
&\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} s(\widetilde{MVR}(i)) \\
&\quad \geq \frac{1}{N_{\text{card}}} \sum_{i:MVR(i)\in\mathcal{V}} s(\widetilde{MVR}(i)) \text{ (because } score(\cdot) \geq 0) \\
&\quad = \frac{1}{N_{\text{card}}} \Bigg[\sum_{i:MVR(i)\in\mathcal{V}} a^{\text{PZ}}_{\text{POLL}}(CVR(i), MVR(i)) \\
&\qquad + \sum_{i:MVR(i)\notin\mathcal{V}} a^{\text{PZ}}_{\text{POLL}}(CVR(i), MVR(i)) \Bigg] \\
&\qquad \text{(by Definition 23—the second sum is zero)} \\
&\quad = \frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} a^{\text{PZ}}_{\text{POLL}}(CVR(i), MVR(i)) \\
&\quad > 1/2
\end{aligned}$$

So $\widetilde{MVR}$ is a counterexample showing that s does not satisfy Definition 22. □

For card-level comparison audits, we can also build a conservative assorter.

Suppose that s is a function from $\mathcal{V}$ to $[0, u_s]$ that satisfies Equation (6). Define the phantoms-to-zombies transformation for comparison audits as follows.

Examples of BAD ballot types are:

- $\text{FIND}(i) = \perp$,
- $\text{FIND}(i) = id$ but the card that was returned did not have the correct ID,
- no card was returned, or
- there was no consensus on what was on the card.

Definition 25. We will extend the scoring function by simply scoring all BAD votes as zero. Cards that do not contain the contest are scored as 0.5. (This is appropriate for non-style based sampling, but will be overruled for style-based

sampling—see Definition 28.) Define the scoring extension $\widetilde{s}$ by:

$$\widetilde{s}(v) = s(v) \text{ if } v \in \mathcal{V}$$
$$\widetilde{s}(v) = 0.5 \text{ if } v = \text{NOCONTEST}$$
$$\widetilde{s}(v) = 0 \text{ if } v \in \text{BAD (not including NOCONTEST)}$$

This obviously has the same upper bound as s.

There are different correct ways of dealing with inconsistencies between the STYLE declaration and the actual presence of the ballot on the contest, which are suited to different sampling strategies.

Definition 26. Define the no-style phantoms-to-zombies comparison assorter as:

$$a^{\text{PZ}}_{\text{COMP-NO-STYLE}}(c, m) = \frac{u_s - (\widetilde{s}(c) - \widetilde{s}(m))}{2u_s - \nu_{\text{NO-STYLE}}} \tag{7}$$

for all $c, m \in \mathcal{V} \cup \text{BAD}$, where $\nu_{\text{NO-STYLE}}$ is the *assorter margin*, $\nu_{\text{NO-STYLE}} = \frac{2}{N_{\text{card}}}[\sum_{i=1}^{N_{\text{card}}} \widetilde{s}(CVR(i))] - 1$.

This considers all cards, regardless of whether the prover claims they contain the contest or not. This is appropriate for sampling strategies that include all ballots.

Proposition 27. *If s satisfies Definition 22 for assertion A then $a^{\text{PZ}}_{\text{COMP-NO-STYLE}}$ as defined in Definition 26 is an assorter that expresses A. That is, it satisfies Equation (1) over CVR-MVR pairs from $\mathcal{V} \cup \text{BAD}$. They have the same upper bound.*

Proof. Very similar to the proof of Proposition 24 and pp. 9–10 of SHANGRLA.

We prove the contrapositive. Suppose that $a^{\text{PZ}}_{\text{COMP-NO-STYLE}}$ does not satisfy Equation (1). Then there must be specific values of *CVR* and *MVR* s.t. $A\left(\{MVR(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false but the LHS of the implication is true, so:

$$\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} a^{\text{PZ}}_{\text{COMP-NO-STYLE}}(CVR(i), MVR(i)) > 1/2.$$

Suppose there is some $v \in \mathcal{V}$ s.t. adding more of v to the MVRs cannot cause A to switch from false to true.

Consider a modified list $\widetilde{MVR}$ of *MVR*s in which v replaces all BAD values. By assumption, $A\left(\{\widetilde{MVR}(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false. We will show that $\widetilde{MVR}$ makes the left side of Equation (6) true. From above:

$$\frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} \frac{u_s - \widetilde{s}(CVR(i)) + \widetilde{s}(\widetilde{MVR}(i))}{2u_s - \nu_{\text{NO-STYLE}}} > 1/2$$

$$\therefore u_s - \frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} (\widetilde{s}(CVR(i)) + \frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} \widetilde{s}(\widetilde{MVR}(i)) >$$
$$u_s - \frac{1}{N_{\text{card}}} \sum_{i=1}^{N_{\text{card}}} \widetilde{s}(CVR(i)) + \frac{1}{2}$$

So the left side of Equation (6) is true but the assertion is false. Hence $\widetilde{MVR}$ is a counterexample showing that s does not satisfy Definition 22. □

Definition 28. Let I be an index set of size N_{contest}.

Define the stylish phantoms-to-zombies comparison assorter as:

$$a^{\text{PZ}}_{\text{COMP-STYLE}}(c, m) = \begin{cases} \frac{u_s - (\widetilde{s}(c) - \widetilde{s}(m))}{2u_s - \nu_{\text{STYLE}}} \\ \text{if } m \text{ contains the contest} \\ \frac{u_s - \widetilde{s}(c)}{2u_s - \nu_{\text{STYLE}}} \\ \text{if } m = \text{NOCONTEST} \end{cases}$$

where $\nu_{\text{STYLE}} = \frac{2}{N_{\text{contest}}}[\sum_{i \in I} \widetilde{s}(CVR(i))] - 1$ is the *assorter margin* over I,

Note that for the style-based assorter, when the *MVR* is NOCONTEST its score is zero. This is important for dealing conservatively with lies about style.

Note that $a^{\text{PZ}}_{\text{COMP-NO-STYLE}}$ and $a^{\text{PZ}}_{\text{COMP-STYLE}}$ use different definitions of the margin.

Remark 29. *$a^{\text{PZ}}_{\text{COMP-STYLE}}$ obviously satisfies Equation (4): when the style information claims the card has the contest, $a^{\text{PZ}}_{\text{COMP-STYLE}}$ always assigns a present, valid MVR at least as high a value as a BAD MVR, or one that does not contain the contest. Indeed, it satisfies the even stronger condition given in the paragraph below Equation (4):*

$$a^{\text{PZ}}_{\text{COMP-STYLE}}(c, \text{NOCONTEST}) = a^{\text{PZ}}_{\text{COMP-STYLE}}(c, m_b)$$

for all $m_b \in$ BAD.

Proposition 30. *If s satisfies Definition 22 for assertion A then $a^{\text{PZ}}_{\text{COMP-STYLE}}$ as defined in Definition 28 satisfies Equation (2) over CVR-MVR pairs from $\mathcal{V} \cup \text{BAD}$.*

Proof. Again, we prove the contrapositive of the proposition. Suppose that $a^{\text{PZ}}_{\text{COMP-STYLE}}$ does not satisfy Equation (2). Then there must be specific values of *CVR* and *MVR* s.t.

$$A\left(\{MVR(i)\}_{i=1}^{N_{\text{card}}}\right) \text{ is false}$$

but the LHS of the implication is true, so:

$$\frac{1}{N_{\text{contest}}} \sum_{i \in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), MVR(i)) > 1/2$$

By assumption, there are at most N_{contest} *MVR*s that contain the contest. Let L be the set of indices of *MVR*s that contain the contest but are not in I (L stands for 'lying' because these are cards that the prover claimed did not have the contest, knowing that the audit would not sample them). Since $|I| = N_{\text{contest}}$ by construction, there must be at least $|L|$ indices in $i \in I$ for which the $MVR(i)$ does not contain the contest.

Consider a modified list MVR_2 in which each item in the set $\{MVR(l) : l \in L\}$ is swapped with a distinct MVR with an index in I that does not contain the contest. Since MVR_2 is a rearrangement of *MVR*s, with no changes, this does not change the value of the assertion.

Hence $A\left(\{MVR_2(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false.

By definition of $a^{\text{PZ}}_{\text{COMP-STYLE}}$ (see Remark 29), this rearrangement does not decrease the value of the assorter on any $MVR(i)$ with $i \in I$. Therefore:

$$\frac{1}{N_{\text{contest}}}\sum_{i\in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), MVR_2(i)) \geq \frac{1}{N_{\text{contest}}}\sum_{i\in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), MVR(i)) > 1/2$$

Again, we use the assumption that there is some $v \in \mathcal{V}$ s.t. changing BAD MVRs to v cannot cause A to switch from false to true.

Consider a modified list $\widetilde{MVR_2}$ of MVRs in which v replaces all BAD values in MVR_2, including any NOCONTEST that remain in I. (NOCONTEST values that are outside I are unchanged.) By assumption, $A\left(\{\widetilde{MVR}_2(i)\}_{i=1}^{N_{\text{card}}}\right)$ is false.

By Remark 29 (i.e., Equation (3)), this change does not decrease the value of the assorter on any $MVR(i)$ with $i \in I$. Therefore:

$$\frac{1}{N_{\text{contest}}}\sum_{i\in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), \widetilde{MVR}_2(i)) \geq \frac{1}{N_{\text{contest}}}\sum_{i\in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), MVR_2(i)) > 1/2$$

By construction, $\widetilde{MVR}_2(i) = \text{NOCONTEST} \iff i \notin I$. Also, $\widetilde{MVR}_2(i) \in \mathcal{V} \iff i \in I$.

We will show that $\widetilde{MVR_2}$ makes the left side of Equation (6) true. From above:

$$1/2 < \frac{1}{N_{\text{contest}}}\sum_{i\in I} a^{\text{PZ}}_{\text{COMP-STYLE}}(CVR(i), \widetilde{MVR}_2(i)) = \frac{1}{N_{\text{contest}}}\sum_{i\in I} \frac{u_s - (\widetilde{s}(CVR(i)) - \widetilde{s}(\widetilde{MVR}_2(i)))}{2u_s - \nu_{\text{STYLE}}}$$

$$\therefore 2u_s - \frac{2}{N_{\text{contest}}}[\sum_{i\in I}\widetilde{s}(CVR(i))] + 1 < \frac{2}{N_{\text{contest}}}\sum_{i\in I}(u_s - \widetilde{s}(CVR(i)) + \widetilde{s}(\widetilde{MVR}_2(i))) \text{ (definition of } \nu_{\text{STYLE}})$$

$$\therefore \frac{2}{N_{\text{contest}}}\sum_{i\in I}\widetilde{s}(\widetilde{MVR}_2(i))) > 1 \text{ (because } |I| = N_{\text{contest}})$$

$$\therefore \frac{2}{N_{\text{contest}}}\sum_{i\in I} s(\widetilde{MVR}_2(i))) > 1/2 \text{ (because } s(v) = \widetilde{s}(v) \text{ when } v \in \mathcal{V})$$

For $i \notin I, \widetilde{MVR}_2(i) = \text{NOCONTEST}$. Hence $\frac{1}{N-N_{\text{contest}}}\sum_{i\notin I} s(\widetilde{MVR}_2(i)) = 1/2$ by Definition 22. Therefore the overall mean $\frac{1}{N_{\text{card}}}\sum_{i=1}^{N_{\text{card}}} s(\widetilde{MVR}_2(i)) > 1/2$. So $\widetilde{MVR}_2$ is a counterexample showing that s does not satisfy Definition 22.

□

Our main proof relies on using a monotone p-value calculator; see Definition 15.

Monotonicity seems like a desirable property since it implies that the p-value calculator is summarising the evidence numerically in a way that accords with our intuition. Namely, if we shift the observations strictly in a direction that is more consistent with the null hypothesis, then the resulting p-value should increase, and vice versa.

In general, p-value calculators are not required to have this property (it is not a consequence of Definition 13), and indeed some valid p-value calculators are not monotone.

For example, ALPHA used with the truncated shrinkage estimator [21] is not monotone. We can construct counterexamples of the following form: $x_1 = 0$ and $x_2 = 0$, compared to $y_1 = 1$ and $y_2 = 0$. Using ALPHA with the tuning parameters $d = 10$ and $\eta_0 = 0.51$, results in a smaller p-value at timestep $t = 2$ for the x sequence than for the y sequence.

The reason for non-monotonicity in this case is due to how ALPHA (via the truncated shrinkage estimator) is adapting to the earlier observations in the dataset. The initial observation of $y_1 = 1$ makes ALPHA tune itself to be more 'aggressive' at the next observation, and thus it incurs a large 'penalty' when it observes $y_2 = 0$. In contrast, if it started with the initial observation of $x_1 = 0$, ALPHA tunes itself to a more conservative setting and the next observation of $x_2 = 0$ has a smaller impact.

In contrast, it is easy to show that Wald's SPRT and non-adaptive ALPHA (with the η_j tuning parameters set to a constant) are monotone.

The class of p-value calculators is vast, and designing efficient sequentially valid p-value calculators is an active area of research. Based on the above example, we can see that some calculators of current interest will not necessarily be monotone.

It may be possible to relax this requirement and extend our proof to non-monotone calculators. This would allow more flexibility and would therefore likely allow more efficient implementations in practice. Since efficiency is not the primary goal of our current paper, we leave such extensions as potential future work.